

PROYECTO: Gimnasio 'Sport'

Trabajo Integrador N°1

GRUPO FIORITECH:

Facundo Lella – Legajo: 14467

Ramiro Martinez – Legajo: 14334

Alejandro Moreno – Legajo: 14336

Renzo Dávila – Legajo: 14325

Profesores:

Lucias Cortez

Leandro Spadaro

2024



Índice

1. Metodología de desarrollo	4
2. Iniciación	4
2.1. Modelado de Negocio	4
2.2. Requisitos	6
2.2.1. Requisitos funcionales	6
2.2.2. Casos de uso	6
3. Gestión de Proyecto	7
3.1. Viabilidad	7
3.2. Estimación de tiempos de desarrollo e implementación	7
3.3. Estimación de riesgos	7
3.4. Estimación de costos	7
3.5. Forma de realización del proyecto: Comprar/Construir	7
3.6. Presupuesto	8
3.7. Seguimiento Avance Proyecto	8
4. Entorno	8
4.1. Arquitectura	8
4.2. Tecnología	8
4.3. Base de datos	8
4.4. Tipo de Software	8
4.5. Herramientas	9
5. Elaboración	9
5.1. Requisitos	9
5.2. Requisitos funcionales	10
5.3. Diagrama de casos de uso	10
5.4. Escenarios de casos de uso (Envío mensajes)	11
5.5. Prototipado de pantallas, envió de mensajes	12
5.6. Requisitos no funcionales	13
6. Análisis y diseño	13
6.1. Diagrama de clases de diseño	13
6.2. Diagrama de secuencia de diseño	13
6.3. Diagrama Entidad-Relación de laBase de Datos	14
7. Construcción	14
7.1. Implementación	15
7.2. Framework de trabajo	15
7.3. Base de Datos	15
7.4. Codificación	15
8. Pruebas	16
8.1. Pruebas unitarias	16
8.2. Pruebas de interfaz	16
9. Transición	16

10.Propuesta de Mejora	16
10.1. Referencia de mercado	16
10.2. Propuesta de nuevas funcionalidades	17
10.3. Modificación del diagrama de clases	17
11.Propuesta de refactorización	18
11.1. Vista	18
11.2. Controlador	18
11.3. Modelo	19
11.4. Acceso a datos	19
12.Material de Exposición en Conferencia	20
12.1. Porcentaje de avance del software	20
12.2. Powerpoint presentación	20
13.Anexos	20
13.1. Anexo 1	20
13.2. Anexo 2	20
13.3. Anexo 3	20
13.4. Anexo 4	20

1. Metodología de desarrollo

La metodología que se utilizó para la realización del proyecto fue RUP (proceso unificado racional), la cual es una metodología iterativa conformada por las siguientes 4 etapas:

- **Iniciación**, donde se definen el alcance, los objetivos y la visión del sistema.
- **Elaboración**, en la que se modelan los requisitos clave y se diseña la arquitectura base.
- **Construcción**, centrada en la implementación, codificación y pruebas de los componentes.
- **Transición**, donde el sistema es entregado a los usuarios finales y se asegura su correcto funcionamiento en el entorno real.

Además, RUP se apoya en disciplinas transversales como el modelado de negocios, la gestión de requisitos, el diseño, la implementación, las pruebas y la gestión de proyectos, lo que permitió mantener un marco organizado, minimizar riesgos y garantizar que el producto evolucionara de forma controlada y alineada con los objetivos planteados.

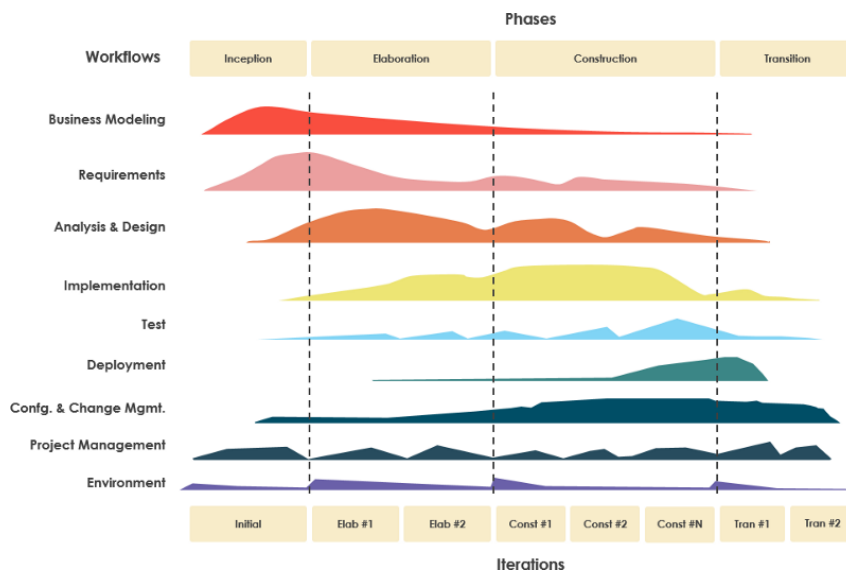


Figura 1: Metodología RUP

2. Iniciación

2.1. Modelado de Negocio

Especificación del Proyecto:

El proyecto consiste en una aplicación web pensada para la administración de gimnasio, teniendo en cuenta tareas administrativas, actividades físicas y pagos en diferentes sucursales. Existirán diferentes tipos de usuarios, cada uno con distintas responsabilidades; entre estos se encuentran los socios, empleados y el administrador de sistema. Se pide

una atención personalizada para cada socio, con mensajes y recordatorios sobre sus pagos o fechas especiales, seguimiento de rutinas para que cada profesor pueda ver el avance de los socios asignados, y variedad en métodos de pago, todo esto para hacer la experiencia del usuario más satisfactoria.

Entrevista con el Cliente

A la hora de la captura de requisitos, surgieron preguntas sobre la estructura del proyecto, algunas de las más importantes fueron:

- Los diferentes empleados, ¿tendrán distintos permisos? ¿cada uno podrá acceder a pestañas específicas?. No, todo empleado tendrá las mismas posibilidades, por si tiene que tomar el rol de otro empleado por un instante.
- El envío de mensajes, ¿sera un envío general para un numero de socios?. Si, se busca hacer esta tarea de forma rápida, y generando un mensaje general y enviarlo a todos es lo mejor.
- ¿Que medios de pago se tendrán en cuenta? Los medios de pago disponibles serán en efectivo, transferencia o a través de mercado pago

Diagrama de Clases de Negocio

Se planteó el siguiente diagrama , en donde se ve un poco la estructura que va a tener el sistema.

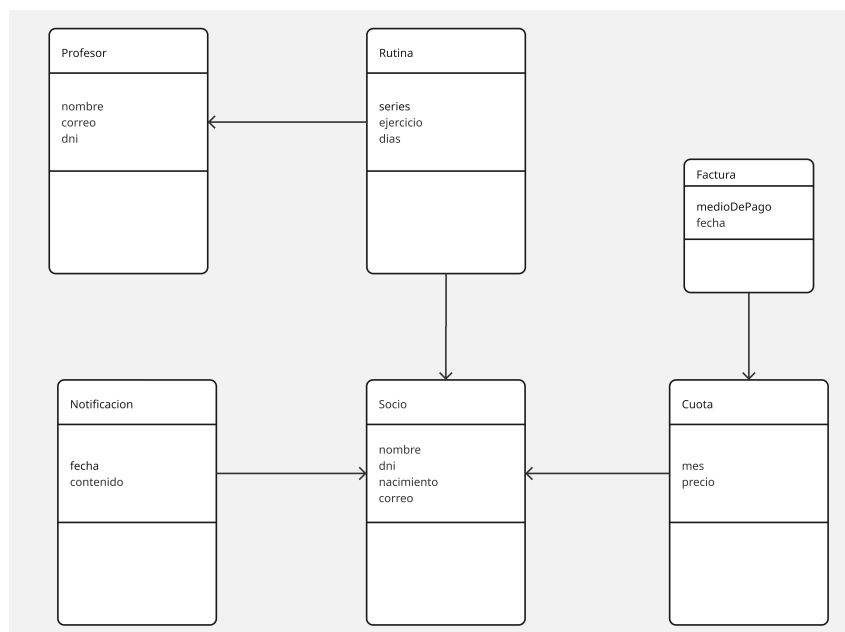


Figura 2: Diagrama de clases de negocio

Modelado de Paquetes del Dominio

Las clases del dominio se acomodan en paquetes, los cuales nos ayudarán a comprender el problema, además de empezar a ver la organización que tendrán las clases del proyecto.

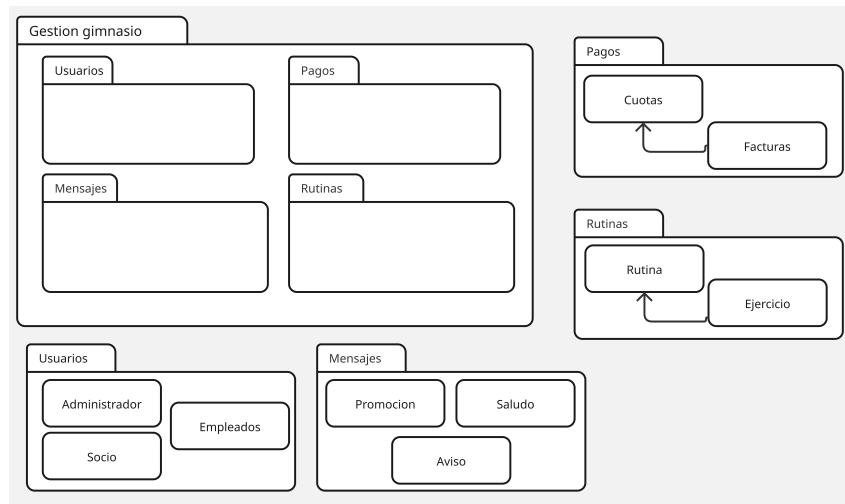


Figura 3: Diagrama de paquetes de dominio

2.2. Requisitos

2.2.1. Requisitos funcionales

Se definieron los requisitos funcionales, que se pudieron extraer de la especificación del proyecto; se identificaron los siguientes:

- Administración de los empleados, por parte del administrador del sistema.
- Dar alta nuevos socios, por parte de los empleados.
- Enviar mensajes de diferentes tipos (avisos, saludo, promociones).
- Permitir a los socios pagar la cuota mensual con Mercado Pago.
- Los socios podrán llevar seguimiento de sus rutinas, las cuales serán proporcionadas por los profesores.

2.2.2. Casos de uso

Se definieron los casos de uso según su importancia y prioridad; son los siguientes:

CASOS DE USO		
Críticos	Medio	Bajo
ABM Valor Cuota y ABM Cuota Mensual	ABM Empleados	Login
Pago de Cuota con su generación de factura y forma de pgo.	ABM Socios	
ABM Rutina	ABM Sucursal	
Mostrar Detalle de Deuda y Mostrar todas las facturas pagadas de un mes		
Completar Detalle Rutinas Realizadas, Mostar Rutinas incumplidas y mostrar porcentaje cumplido de Rutina		
Envío Deuda, Saludos de Cumpleaños y Cuota Mensual por Correo		

Figura 4: Casos de uso

3. Gestión de Proyecto

3.1. Viabilidad

El proyecto se considera viable en virtud de que el equipo ha adquirido los conocimientos técnicos y metodológicos necesarios para garantizar su adecuada implementación. Asimismo, resulta factible para el cliente dado que se trata del primer desarrollo formal del equipo, lo cual implica la adopción de estándares de precios reducidos respecto del mercado. De este modo, se asegura no solo la correcta ejecución del sistema, sino también una propuesta económicamente accesible que favorece la relación costo-beneficio para el cliente.

3.2. Estimación de tiempos de desarrollo e implementación

Para el desarrollo del proyecto se estimó un tiempo de 52 días, empezando el 4 de agosto hasta el 25 de septiembre. En este tiempo se considera tanto el aprendizaje de las tecnologías como el recorrido de las diferentes etapas de RUP.

Además, se dividieron las tareas de diseño y desarrollo entre el equipo de desarrollo.

3.3. Estimación de riesgos

En el análisis de viabilidad se llevó a cabo una estimación de riesgos considerando tanto factores internos como externos que podrían afectar el desarrollo del proyecto. Entre ellos se contemplaron las responsabilidades académicas y personales del equipo, las cuales podrían generar demoras en determinadas etapas, así como eventuales inconvenientes de carácter imprevisto, tales como enfermedades u otras situaciones que impacten en la disponibilidad de los integrantes. Estas contingencias fueron evaluadas con el propósito de establecer márgenes de flexibilidad en la planificación, garantizando que el proyecto pueda sostenerse aun ante la ocurrencia de tales eventualidades.

3.4. Estimación de costos

En lo que respecta a la estimación de costos, se tomaron como referencia los valores orientativos establecidos por el Consejo Profesional de Ciencias Informáticas de Córdoba (CPCI), los cuales constituyen un parámetro reconocido para la determinación de honorarios en proyectos de desarrollo de software. No obstante, considerando que este trabajo corresponde a las primeras experiencias formales del equipo de desarrollo, se decidió aplicar una reducción significativa respecto de dichos valores de referencia. Esta estrategia no solo permite ofrecer al cliente una solución de calidad profesional a un costo accesible, sino que también representa una oportunidad valiosa para el equipo, en tanto posibilita avanzar en su carrera profesional mediante la adquisición de experiencia práctica y el fortalecimiento de su trayectoria en el ámbito del desarrollo de software.

3.5. Forma de realización del proyecto: Comprar/Construir

En relación con la forma de realización del proyecto, este fue diseñado íntegramente desde cero, lo que permitió garantizar que cada uno de sus componentes respondiera de manera precisa a los requerimientos identificados durante la etapa de revelamiento. El equipo contaba con las competencias técnicas y metodológicas necesarias para llevar a

cabo el desarrollo completo, lo que posibilitó una solución personalizada y ajustada a las necesidades específicas del cliente. Este enfoque integral no solo aseguró la coherencia entre el diseño y la implementación, sino que también fortaleció la capacidad del equipo para abordar proyectos de mayor complejidad en el futuro.

3.6. Presupuesto

Se incluye el link al presupuesto planteado en Anexo 1

3.7. Seguimiento Avance Proyecto

El seguimiento de avance del proyecto se llevaba a cabo en reuniones semanales con el cliente, en donde se hablaba del progreso del desarrollo y se solucionaban dudas que surgían en el camino. Estas reuniones se realizaron los días martes de 19:30 a 20:30.

4. Entorno

4.1. Arquitectura

El desarrollo del proyecto incluyo el uso del patrón MVC (Modelo, Vista, Controlador), el cual ayuda a la organización del equipo, mejor mantenimiento y escalabilidad.

4.2. Tecnología

- Lenguaje de programacion: **Java**
- Kit de desarrollo: **JDK 17.0.16**
- Entorno de ejecucion: **JRE 17.0.16**
- Base de datos: **MySQL 8.0.43**
- IDEs para desarrollo: **Visual Studio Code, NetBeans e IntelliJ IDEA**
- Framework de interfaces web: **Thymeleaf**

4.3. Base de datos

Para la base de datos se utilizo MySQL 8.0.43, también se utilizaron herramientas de visualización de la base de datos como DBeaver y DataGrip, esta ultima ayudando a la creación del DER.

4.4. Tipo de Software

El tipo de software desarrollado es una aplicación web, esto ya que el cliente solicitó específicamente este tipo de desarrollo. Además, el mercado actual está enfocado en aplicaciones web, dejando atrás a las de escritorio.

4.5. Herramientas

Comunicación del equipo

Para la comunicación se usaron dos medios principales: **WhatsApp**, para el contacto constante y avisos de importancia, y **Discord**, para la realización de reuniones en horario de desarrollo.

Captura de requerimientos

No se ocuparon herramientas de captura de requisitos, ya que en esta ocasión el cliente dio entrega de un documento con los requisitos y funcionalidades bien definidos.

Prototipado de interfaces gráficas UI

Para el prototipado de interfaces gráficas se utilizó **Balsamiq**, esta nos permitió realizar prototipos simples pero útiles para poder lograr una mejor comprensión de los requerimientos del sistema.

Modelado

Para el diseño del modelo se usaron las herramientas de **Miro** y **UMLetino**, la primera permitiendo un ambiente colaborativo.

Codificación

La codificación del proyecto tuvo lugar en tres IDEs diferentes: **NetBeans**, **Visual Studio Code** y **IntelliJ**, contando los dos últimos con agentes de codificación como son Codex y Junie respectivamente. Además se usaron herramientas como Code With Me en IntelliJ para crear un ambiente colaborativo en tiempo real, y un versionador de código, en este caso **Git** junto con **GitHub**.

Pruebas de software

Las pruebas se hicieron a través del código, creando pequeños unittest para evaluar las funcionalidades, y una vez terminada probando manualmente con la interfaz gráfica.

Gestión de Proyecto

Para la gestión del proyecto únicamente se usó WhatsApp para planificar el desarrollo del mismo, y GitHub para su versionado.

5. Elaboración

Esta fase se enfoca en refinar y detallar los requisitos y el entendimiento del negocio que adquirimos en la fase de iniciación.

5.1. Requisitos

En esta etapa ya se hizo un mejor análisis del problema, por lo que se encontraron aquellos requisitos que no se especifican en el problema pero son necesarios para el correcto funcionamiento del sistema, se listaron los requisitos anteriores junto con los nuevos

5.2. Requisitos funcionales

- Administración de los empleados, por parte del administrador del sistema.
- Dar alta nuevos socios, por parte de los empleados.
- Enviar mensajes de diferentes tipos (avisos, saludo, promociones).
- Permitir a los socios pagar la cuota mensual con Mercado Pago.
- Los socios podrán llevar seguimiento de sus rutinas, las cuales serán proporcionadas por los profesores.
- Dar de alta de Cuotas, Empleados, Socios, Mensajes y Rutinas.
- Dar de baja cuotas, empleados, socios, mensajes y rutinas
- Poder modificar empleados, socios, precios de cuota, rutinas y mensajes.

5.3. Diagrama de casos de uso

Se muestra un diagrama de los casos de uso del sistema

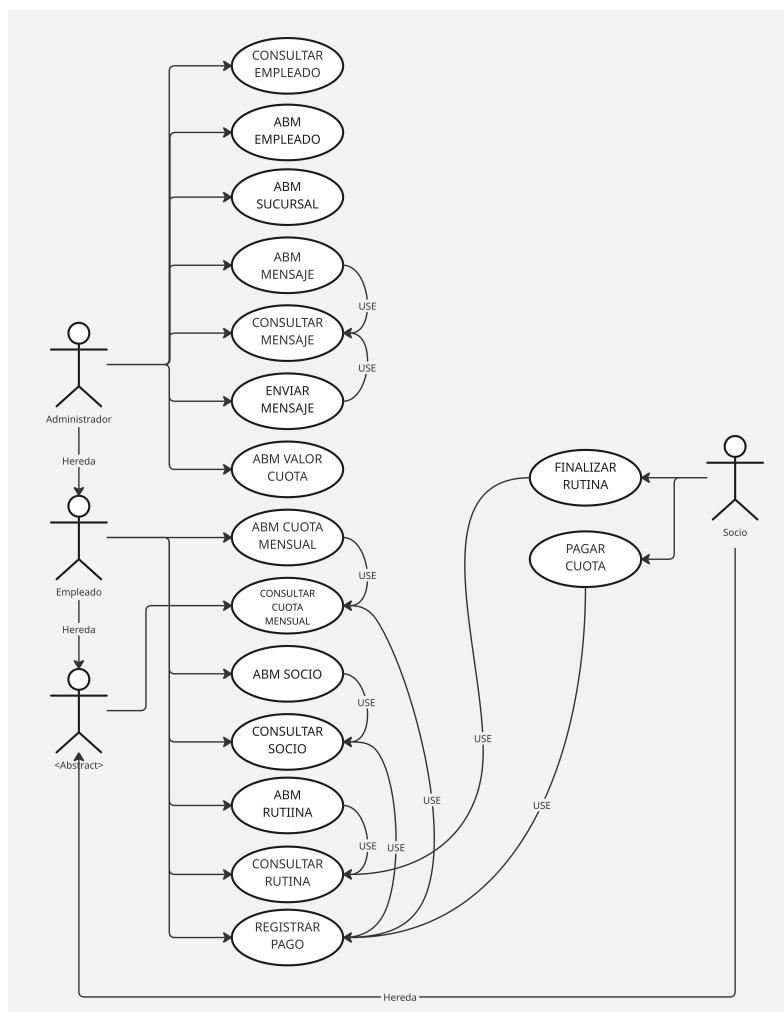


Figura 5: Diagrama de casos de uso

5.4. Escenarios de casos de uso (Envío mensajes)

Envío de mensajes	
Descripción	El sistema permite crear y enviar mensajes personalizados a los socios del gimnasio, de acuerdo con el tipo de mensaje definido.
Requisitos especiales	Conexión a internet.
Precondiciones	Tener un usuario autenticado con permisos de empleado o administrador. Contar con datos de socios registrados en el sistema. Disponer de la información necesaria para el tipo de mensaje a enviar (cuotas, deudas o fechas de cumpleaños).
Poscondiciones	Los mensajes se enviarán a los destinatarios correspondientes según el tipo de mensaje configurado.
Activación	El usuario selecciona la opción “Mensajes” en el sistema.
Flujo principal	1- El usuario selecciona la opción “Crear mensaje”. 2- El sistema solicita el tipo del mensaje a enviar (ENVÍO CUOTA, ENVÍO DEUDAS, SALUDO CUMPLEAÑOS). 3- El usuario completa los datos y presiona el botón “Crear mensaje”. 4- El sistema almacena el mensaje en la lista de mensajes pendientes. 5- El usuario accede a la lista de mensajes creados. 6- El usuario selecciona un mensaje y presiona el botón “Enviar”. 7- El sistema procesa el envío: – Para mensajes de cuota, los envía a los usuarios que poseen una cuota impagada que no ha vencido. – Para mensajes de deudas, los envía únicamente a los socios con saldo pendiente. – Para saludos de cumpleaños, los envía únicamente a los socios que cumplan años en la fecha actual. 8- El sistema confirma que el mensaje fue enviado exitosamente.
Flujo alternativo	3.1- El usuario cancela la creación del mensaje → fin del caso de uso. 6.1- El usuario cancela el envío → se mantiene el mensaje en la lista de pendientes.
Puntos de extensión	5.1- El usuario selecciona la opción “Modificar mensaje” antes de enviarlo. 5.1.1- El sistema abre el formulario de edición de mensaje y guarda los cambios.

5.5. Prototipado de pantallas, envió de mensajes

Se realizo un prototipo de pantalla para el caso de uso critico asignado, esto con el fin de mostrar al cliente como sera el sistema y como interactuara con el.

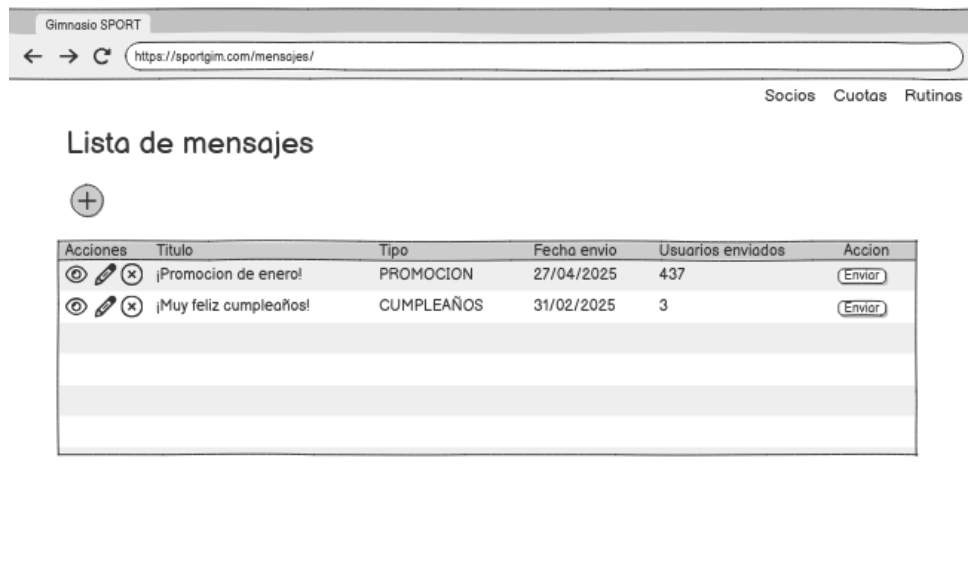


Figura 6: Lista de mensajes creados



Figura 7: Pestaña de creación/edición de mensajes

5.6. Requisitos no funcionales

Categoría	Requisito
Rendimiento	El sistema deberá permitir que los socios puedan acceder a su rutina y estado de pagos en menos de 3 segundos desde que se ingresa la solicitud.
Disponibilidad	La aplicación deberá estar disponible todos los días a todas horas.
Seguridad	El sistema deberá autenticar a todos los usuarios mediante credenciales seguras, y los datos de pagos deberán transmitirse en forma cifrada.
Usabilidad	La interfaz deberá ser sencilla e intuitiva, permitiendo que un nuevo usuario pueda comprender su uso básico en menos de 15 minutos.
Mantenibilidad	El sistema deberá estar documentado y estructurado para facilitar la corrección de errores y la incorporación de nuevas funcionalidades.
Compatibilidad	El acceso al sistema deberá ser posible desde navegadores modernos como Google Chrome y Mozilla Firefox en sus últimas versiones.
Escalabilidad	El sistema deberá soportar un incremento en la cantidad de socios de hasta un 50 % sin necesidad de rediseñar la arquitectura base.

6. Análisis y diseño

6.1. Diagrama de clases de diseño

Para el desarrollo del proyecto se elaboró inicialmente un diagrama de clases de diseño por el equipo, con el objetivo de capturar las perspectivas y propuestas de modelado. Posteriormente, los diagramas de cada equipo fueron revisados de manera conjunta y unificados en un único modelo consensuado, que representa la visión final acordada entre todos los desarrolladores.

Con el fin de facilitar su análisis y manipulación, el diagrama de clases definitivo se encuentra disponible en el Anexo 2.

6.2. Diagrama de secuencia de diseño

Se realizó el diagrama de secuencia del caso de uso crítico asignado, con el fin de visualizar la lógica de código de una manera más sencilla.

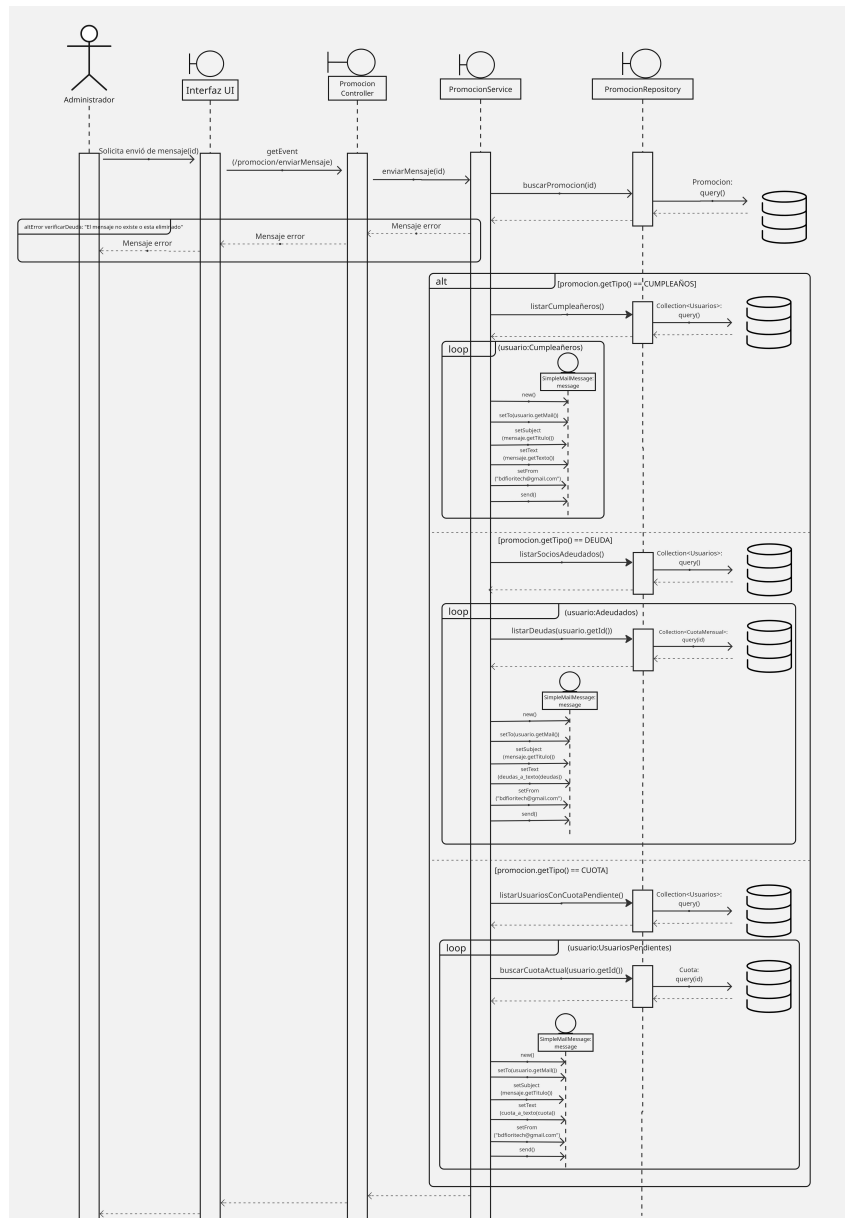


Figura 8: Diagrama de secuencia

6.3. Diagrama Entidad-Relación de la Base de Datos

Se obtuvo el diagrama entidad relacion con el fin de poder visualizar como se relacionan los objetos que persisten en la base de datos. Esto ayuda a representar de manera clara y estructurada la información que será gestionada por el sistema.

En este caso el diagrama fue generado por DataGrip, gestor de base de datos de JetBrains.

El archivo esta presentado en el Anexo 3

7. Construcción

Esta etapa trata principalmente de la implementación de las distintas partes del sistema para lograr un incremento funcional del mismo. Es decir, esta etapa trata princi-

palmente de codificar todo lo que se logró del análisis de los requisitos en las anteriores etapas. Recordemos que si bien la codificación pudo ser empezada mínimamente en la etapa de elaboración, es esta en la cual será el foco principal.

7.1. Implementación

7.2. Framework de trabajo

El proyecto se desarrolló sobre **JDK 17.0.16**, utilizando **Spring Boot** para la gestión de la aplicación y **Thymeleaf** como motor de plantillas para la integración entre vistas HTML y lógica de control. Cada integrante trabajó con su IDE de preferencia (**IntelliJ IDEA**, **NetBeans** o **Visual Studio Code**), y en caso de dificultades se realizaban reuniones para resolver problemas en conjunto.

Además, se empleó **Git** junto con **GitHub** como sistema de control de versiones, lo que permitió llevar un historial de cambios, facilitar el trabajo colaborativo y asegurar la correcta integración del código durante todo el desarrollo.

7.3. Base de Datos

Para la persistencia de los datos del sistema se optó por utilizar el gestor **MySQL**, dado que ofrece un alto rendimiento, amplia documentación y compatibilidad con múltiples entornos de desarrollo. La integración con la aplicación se realizó mediante el módulo **Spring Data JPA**, el cual permitió gestionar las conexiones y operaciones sobre la base de datos a través de anotaciones, simplificando la configuración y reduciendo la necesidad de código repetitivo. Gracias a esta elección, se logró una comunicación fluida entre el back-end y la base de datos, garantizando la integridad y consistencia de la información en el sistema del Gimnasio *Sport*.

7.4. Codificación

La etapa de codificación se llevó a cabo siguiendo el patrón **Modelo–Vista–Controlador** (MVC), lo que permitió mantener una separación clara de responsabilidades dentro del sistema. En particular, el **modelo** se estructuró en tres niveles:

- Dominio: representación de las entidades principales del sistema mediante clases anotadas con **@Entity**.
- Servicios: capa encargada de la lógica de negocio, definida a través de clases con la anotación **@Service**.
- Repositorios: gestión de la persistencia de datos utilizando interfaces extendidas de **JpaRepository**, anotadas con **@Repository**.

La **vista** se implementó con el motor de plantillas **Thymeleaf**, que permitió generar páginas HTML dinámicas vinculadas al back-end. Finalmente, los **controladores**, anotados con **@Controller**, gestionaron la interacción entre usuarios, servicios y vistas, asegurando una comunicación fluida dentro del sistema.

El código completo del proyecto se encuentra disponible en el repositorio oficial de GitHub, el cual está vinculado en el Anexo 4

8. Pruebas

8.1. Pruebas unitarias

Durante el desarrollo se realizaron **pruebas unitarias** con el fin de verificar el correcto funcionamiento de los distintos módulos del sistema. Para ello se implementaron casos de prueba hardcodeados, diseñados específicamente para simular diferentes escenarios de uso y validar que los resultados obtenidos coincidieran con los esperados. Esta estrategia permitió detectar errores de manera temprana y asegurar la confiabilidad de las funciones principales antes de su integración con el resto de la aplicación.

8.2. Pruebas de interfaz

Los casos definidos en las pruebas unitarias fueron replicados en la **interfaz gráfica**, con el objetivo de evaluar la correcta conexión entre el front-end desarrollado con Thymeleaf y el back-end implementado en Spring Boot. Esta etapa permitió comprobar que los datos procesados en la lógica de negocio se reflejaran adecuadamente en las vistas y que las acciones de los usuarios desencadenaran las operaciones esperadas en el sistema, garantizando así la coherencia entre ambas capas de la aplicación.

9. Transición

La etapa de **Transición** se orienta a la entrega del producto al cliente y a su puesta en funcionamiento en el entorno real. En un proyecto de pequeña escala como el presente, esta fase se centra principalmente en realizar pruebas finales con los usuarios, capacitar al personal en el uso de la aplicación y asegurar que la instalación del sistema se lleve a cabo sin inconvenientes. Asimismo, se brinda un período corto de soporte inicial para resolver posibles incidencias, garantizando que el software cumpla con los objetivos establecidos y quede plenamente operativo para su utilización diaria.

10. Propuesta de Mejora

10.1. Referencia de mercado

En el mercado existen plataformas consolidadas como Glofox, GymMaster y otras soluciones globales de gestión para gimnasios que ofrecen funcionalidades avanzadas para optimizar la operación, retención de miembros y comunicación con los usuarios. Por ejemplo, Glofox permite organizar horarios de clases, manejar afiliaciones, procesar pagos, enviar campañas automáticas y realizar análisis en tiempo real del desempeño del gimnasio.

Por su parte, GymMaster incorpora control de acceso 24/7, facturación integrada, aplicaciones móviles para miembros, gestión de punto de venta y herramientas de marketing automatizadas.

Estas plataformas ofrecen un nivel de madurez funcional que les permite atender necesidades amplias de gimnasios con múltiples sucursales, clases grupales, variabilidad de membresías, analíticas avanzadas y automatización.

10.2. Propuesta de nuevas funcionalidades

- App móvil o portal para socios: permitir que los asociados puedan reservar clases, visualizar su rutina, recibir notificaciones, modificar datos desde el celular.
- Control de acceso automatizado: integración con dispositivos físicos (tarjetas, lectores, Bluetooth) para que los socios puedan entrar al gimnasio sin intervención manual.
- Análisis y reportes de métricas: dashboards con indicadores clave (retención de socios, ingresos, uso de rutinas) para que los administradores tomen decisiones informadas.
- Automatización y segmentación de campañas: generar mensajes promocionales personalizados según comportamiento de los socios (por ejemplo, enviar promociones a quienes dejaron de asistir, recomendaciones de rutina, etc.).
- Módulo de punto de venta / ventas propias: vender suplementos, equipamiento, servicios extras dentro del gimnasio y llevar inventario.
- Reservas online y programación de clases / actividades: permitir que los socios se inscriban en clases, monitorear cupos, esperar lista de espera, etc.
- Escalabilidad multi-sucursal: que el sistema pueda gestionar varias sedes con roles particulares y compartir datos de manera centralizada.

10.3. Modificación del diagrama de clases

Para esta sección se tomara en cuenta la funcionalidad de agregar un modulo de venta de productos.

Para incorporarlo se añadieron nuevas entidades al modelo de dominio que permiten gestionar la venta de suplementos, equipamiento y servicios adicionales dentro del gimnasio. De esta forma, se definió la entidad *Producto* para representar los artículos disponibles, *Venta* como registro de cada transacción y *DetalleVenta* para vincular los productos con cada operación. Estas clases se relacionan con una clase ya existente, de modo que un *Socio* puede realizar compras, asegurando la correcta trazabilidad e integración con el sistema.

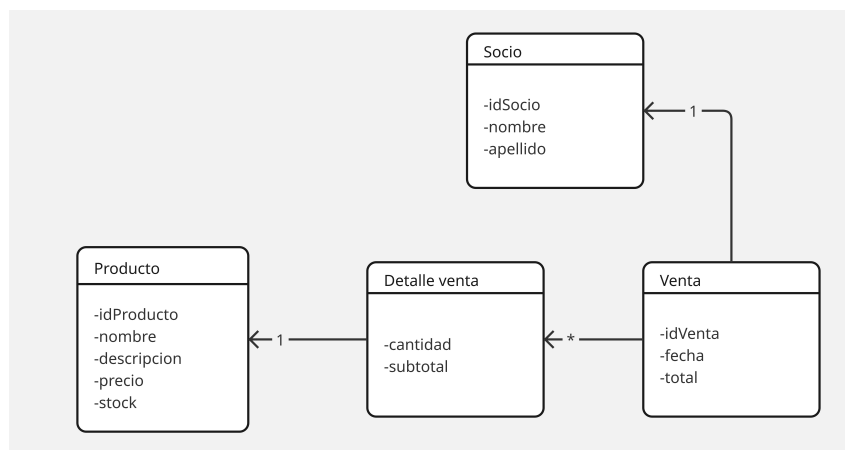


Figura 9: Diagrama de clases de dominio del nuevo modulo

11. Propuesta de refactorización

11.1. Vista

En la capa de vista, la propuesta de refactorización se orienta a mejorar la estética y la usabilidad de la aplicación. Si bien las interfaces actuales desarrolladas con **Thymeleaf** cumplen con los requerimientos funcionales, se plantea la posibilidad de optimizarlas mediante el uso de **plantillas gráficas reutilizables** o, incluso, con la colaboración de un **diseñador especializado** que aporte una identidad visual más atractiva y profesional.

Asimismo, se considera necesario reforzar la experiencia del usuario a través de la incorporación de **hojas de estilo CSS personalizadas**, que unifiquen la paleta de colores, las tipografías y la disposición de los elementos, así como el uso de **JavaScript** para añadir dinamismo e interactividad a las pantallas. Estas mejoras favorecerían la navegación de los socios, empleados y administradores, logrando que las funcionalidades —como la gestión de rutinas, pagos o mensajes— resulten más claras e intuitivas de utilizar.

11.2. Controlador

En la capa de controladores, la propuesta de refactorización se centra en reducir la duplicación de código y mejorar la mantenibilidad del sistema. Actualmente, varios controladores gestionan operaciones muy similares, como el alta, baja, modificación o listado de entidades (socios, empleados, rutinas, productos del punto de venta). Esta situación genera redundancia y hace más complejo el mantenimiento a largo plazo.

Una posible solución consiste en implementar un **controlador genérico**, capaz de manejar estas operaciones comunes mediante el uso de herencia o parametrización. De esta manera, los controladores específicos solo deberían extender el comportamiento base cuando se requiera lógica particular, como el envío de mensajes personalizados a los socios o el cálculo de deudas pendientes.

11.3. Modelo

En la capa de modelo, una de las propuestas de refactorización más relevantes consiste en el uso de **anotaciones de validación directamente en las entidades**. Actualmente, muchas de las comprobaciones sobre los datos (como verificar que un campo no sea nulo, que un correo tenga el formato correcto o que un valor numérico sea mayor a cero) suelen implementarse de forma manual en la lógica de los servicios. Esta práctica genera duplicación de código y dispersión de las reglas de negocio.

La adopción de anotaciones proporcionadas por el estándar **Jakarta Bean Validation** (por ejemplo, `@NotNull`, `@Email`, `@Min`, `@Size`) permitiría que las restricciones se definan de manera declarativa en las propias clases de dominio. De esta forma, las reglas de validación quedarían vinculadas directamente con los atributos que afectan, reduciendo errores, simplificando el mantenimiento y garantizando que las verificaciones se apliquen de forma consistente en todos los puntos del sistema.

Por ejemplo, en la entidad *Socio* podría anotarse el campo de correo con `@Email`, el nombre con `@NotNull` y la edad con `@Min(18)`, lo cual asegura automáticamente que un socio registrado siempre cumpla con las condiciones mínimas requeridas. Esta refactorización no solo incrementa la claridad del modelo, sino que también mejora la robustez general del sistema, facilitando futuras extensiones o integraciones.

11.4. Acceso a datos

Una de las principales ventajas de utilizar **Spring Data JPA** en la capa de acceso a datos es la posibilidad de definir **consultas personalizadas** sin necesidad de escribir manualmente sentencias SQL. Esto se logra mediante los llamados *query methods*, los cuales se generan automáticamente a partir de la convención en el nombre de los métodos dentro de las interfaces de repositorio.

De esta forma, es posible crear consultas específicas siguiendo un esquema declarativo. Por ejemplo, en el contexto del Gimnasio *Sport*, se pueden definir métodos como `findByDeudaGreaterThan(Double monto)` para obtener todos los socios cuya deuda supere un valor determinado, o `findByFechaNacimiento(LocalDate fecha)` para listar a los socios que cumplen años en una fecha dada. Spring Data JPA interpreta el nombre del método y construye dinámicamente la consulta correspondiente sobre la base de datos, reduciendo código y mejorando la legibilidad.

En conjunto, las consultas personalizadas permiten optimizar la capa de persistencia, hacerla más expresiva y adaptada a las necesidades concretas del sistema, al mismo tiempo que mantienen la simplicidad y coherencia del enfoque de Spring Data JPA.

12. Material de Exposición en Conferencia

12.1. Porcentaje de avance del software

Consideramos que el software cumple con el 90 % de las especificaciones, faltando adoptar algún diseño de página más estético.

12.2. Powerpoint presentación

Presentación

13. Anexos

13.1. Anexo 1

Presupuesto del proyecto

13.2. Anexo 2

Para visualizar el diagrama de clases de diseño se deberá descargar el siguiente archivo:

Diagrama de clases de diseño

Para luego entrar e importar el archivo descargado en Umletino:

Umletino

13.3. Anexo 3

Diagrama entidad relación de la base de datos.

13.4. Anexo 4

Repositorio de Github y ubicación del proyecto